

Trabajo Práctico 3

System Programming - Crystal Campus

Organización del Computador 2

Primer Cuatrimestre 2019

1. Objetivo

Este trabajo práctico consiste en un conjunto de ejercicios en los que se aplican de forma gradual los conceptos de *System Programming* vistos en las clases teóricas y prácticas. Los ejercicios están inspirados en un juego para DOS denominado *Crystal Caves* (Fig. 1).

Se busca construir un sistema mínimo que permita correr como máximo 20 tareas concurrentemente a nivel de usuario. Este sistema simulará un juego entre dos jugadores, A y B, sobre un tablero donde se ubicarán tareas. Estas serán creadas a partir de la acción de los jugadores y se moverán sobre el tablero de forma autónoma buscando cristales. Una vez que junten una cierta cantidad volverán a su base para dejar los cristales, cambiarlos por puntos y luego comenzar nuevamente. El jugador que logre obtener la máxima cantidad de puntos será el ganador. Cada tarea o “minero”, será capaz de moverse por el tablero y juntar cristales. Adicionalmente podrá consultar un identificador para reconocer qué número de tarea es y poder diferenciar entre sí las tareas de cada jugador. Sobre el tablero hay cristales de diferentes pesos. Al moverse un minero puede o no juntar un cristal. Esta acción no suma puntos para el jugador hasta que el minero no lleve los cristales a su base.

Los ejercicios de este trabajo práctico proponen utilizar los mecanismos que posee el procesador para la programación desde el punto de vista del sistema operativo.



Figura 1: Crystal Caves: <https://www.playdosgames.com/play/crystal-caves/>

2. Introducción

Para este trabajo se utilizará como entorno de pruebas el programa *Bochs*. El mismo permite simular una computadora IBM-PC compatible desde el inicio, y realizar tareas de debugging. Todo el código provisto para la realización del presente trabajo está ideado para correr en *Bochs* de forma sencilla.

Una computadora al iniciar comienza con la ejecución del POST y el BIOS, el cual se encarga de reconocer el primer dispositivo de booteo. En este caso dispondremos de un *Floppy Disk* como dispositivo de booteo. En el primer sector de dicho *floppy*, se almacena el *boot-sector*. El BIOS se encarga de copiar a memoria 512 bytes del sector, a partir de la dirección 0x7C00. Luego, se comienza a ejecutar el código a partir esta dirección. El boot-sector debe encontrar en el *floppy* el archivo `kernel.bin` y copiarlo a memoria. Éste se copia a partir de la dirección 0x1200, y luego se ejecuta a partir de esa misma dirección. En la figura 3 se presenta el mapa de organización de la memoria utilizada por el *kernel*.

Es importante tener en cuenta que el código del *boot-sector* se encarga exclusivamente de copiar el *kernel* y dar el control al mismo, es decir, no cambia el modo del procesador. El código del *boot-sector*, como así todo el esquema de trabajo para armar el *kernel* y correr tareas, es provisto por la cátedra.

Los archivos a utilizar como punto de partida para este trabajo práctico son los siguientes:

- `Makefile` - encargado de compilar y generar el *floppy disk*.
- `bochsrc` y `bochsdbg` - configuración para inicializar *Bochs*.
- `diskette.img` - la imagen del *floppy* que contiene el *boot-sector* preparado para cargar el *kernel* (*viene comprimida, la deben descomprimir*).
- `kernel.asm` - esquema básico del código para el *kernel*.
- `defines.h` y `colors.h` - constantes y definiciones.
- `gdt.h` y `gdt.c` - definición de la tabla de descriptores globales.
- `tss.h` y `tss.c` - definición de entradas de TSS.
- `idt.h` y `idt.c` - entradas para la IDT y funciones asociadas como `idt_init` para completar entradas en la IDT.
- `isr.h` y `isr.asm` - definiciones de las rutinas para atender interrupciones (*Interrupt Service Routines*).
- `sched.h` y `sched.c` - rutinas asociadas al *scheduler*.
- `mmu.h` y `mmu.c` - rutinas asociadas a la administración de memoria.
- `screen.h` y `screen.c` - rutinas para pintar la pantalla.
- `a20.asm` - rutinas para habilitar y deshabilitar A20.
- `print.mac` - macros útiles para imprimir por pantalla y transformar valores.
- `idle.asm` - código de la tarea *Idle*.
- `game.h` y `game.c` - implementación de los llamados al sistema y lógica del juego.
- `syscalls.h` - interfaz a utilizar en C para los llamados al sistema.
- `taskA.c` y `taskB.c` - código de las tareas (*dummy*).
- `i386.h` - funciones auxiliares para utilizar *assembly* desde C.
- `pic.c` y `pic.h` - funciones `pic_enable`, `pic_disable`, `pic_finish1` y `pic_reset`.

Todos los archivos provistos por la cátedra **pueden** y **deben** ser modificados. Los mismos sirven como guía del trabajo y están armados de esa forma, es decir, que antes de utilizar cualquier parte del código **deben** entenderla y modificarla para que cumpla con las especificaciones de su propia implementación.

A continuación se da paso al enunciado, se recomienda leerlo en su totalidad antes de comenzar con los ejercicios. El núcleo de los ejercicios será realizado en clase, dejando cuestiones cosméticas y de informe para el hogar.

3. Juego

En el presente trabajo práctico deberemos construir un juego de dos jugadores sobre un tablero. El mismo es un rectángulo de celdas de 78 columnas por 40 filas. El lado derecho e izquierdo corresponden a las bases de cada jugador. Estos pueden moverse hacia arriba o abajo sobre las 40 filas de cada lado del tablero. Una vez posicionado en alguna fila, un jugador puede crear una tarea que aparecerá en la columna siguiente o anterior según corresponda. Las tareas o “mineros” operan de forma autónoma, desplazándose por el tablero con el objetivo de juntar cristales.

El tablero donde se ubican las tareas es la representación gráfica de un área de memoria. Cada celda en el tablero representa 8 KB, espacio suficiente para almacenar el código y pila de una tarea. Cuando una tarea se mueve por el tablero, su código es copiado a las celdas dadas por la coordenada donde se movió. Luego se mapea su código a las mismas direcciones virtuales. De esta forma, la tarea continua ejecutando en una posición de memoria física diferente, pero con la misma dirección virtual. En este contexto, puede pasar que dos tareas se muevan a la misma celda, de ocurrir esto, la última que se mueva a la celda pisará por completo el código de la primera.

Los cristales están ocultos en las celdas del tablero. Cada una puede contener un solo cristal con algún peso dado. Las tareas al juntar cristales los guardan uno a uno en un contenedor. Éste es limitado, pudiendo como máximo contener hasta 30 unidades de peso. Si el cristal a juntar por un minero pesa más que el espacio libre su contenedor entonces no podrá guardarse en este. Para descargar los contenedores de los cristales juntados, las tareas deben desplazarse hasta su base (extremo del tablero). Tan sólo con llegar hasta la base los contenedores se descargan. En ese momento es cuando se suman puntos al jugador, correspondientes al peso de cada cristal. El juego termina cuando alguno de los jugadores alcanza los 300 puntos.

Durante el juego, cada jugador podrá tener simultáneamente un máximo de 10 tareas o mineros en el tablero. Si alguna de estas es destruida, podrá crear nuevas tareas hasta un máximo de 20 tareas durante todo el juego.

El tablero por su parte, funciona como la superficie de un cilindro. Tiene dos extremos, lado derecho e izquierdo, uno para cada jugador. Pero es circular en el sentido de arriba-abajo y viceversa. Una tarea en el borde superior puede trasladarse al borde inferior en un salto y viceversa. Este mismo comportamiento puede ser realizado por los cursores de los jugadores.

3.1. Tareas

Las tareas dispondrán de tres servicios, **move**, **take** y **getId**. Estos se atenderán con una única interrupción (47). Los parámetros de cada uno se describen a continuación:

syscall	parámetros	descripción
move	in EAX=177788 in EBX=e_direction_t d	Mueve la tarea en la dirección indicada por d
take	in EAX=310311 out EAX=uint32_t peso	Mina cristales en la celda actual y retorna el peso del cristal obtenido
getId	in EAX=760279 out EAX=uint32_t id	Retorna el id de la tarea actual

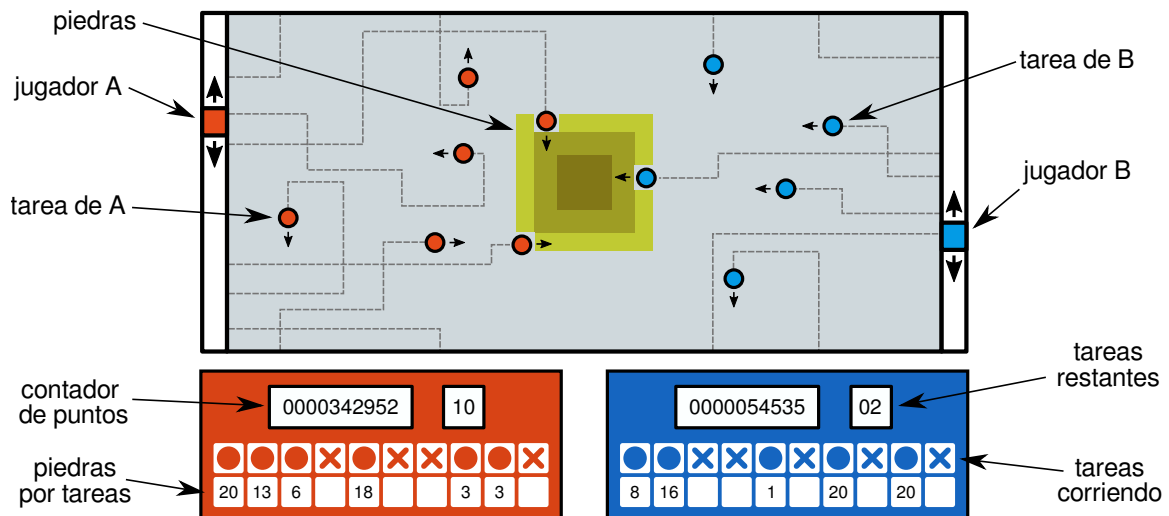


Figura 2: El Juego

Para cualquiera de los servicios de sistema, el registro EAX indica un número que permite identificar al servicio. Si este número no corresponde a ninguno de los servicios la tarea que lo llamo debe ser destruida.

El servicio `move` puede llamarse en cualquier momento. Mediante el registro EBX se indica cual es la dirección a tomar. Las direcciones están codificadas respetando la siguiente enumeración:

```
typedef enum e_direction {
    Right = 1,
    Left = 2,
    Forward = 3,
    Back = 4,
} e_direction_t;
```

Estas direcciones implican diferentes movimientos para cada jugador. Por ejemplo, **Forward** para el jugador de la derecha será moverse hacia la izquierda, mientras que para el jugador de la izquierda será moverse hacia la derecha. Este comportamiento es equivalente para el resto de las direcciones (se encuentran espejadas entre ambos jugadores). La acción de mover no sólo mueve a la tarea dentro del tablero, sino que también copia su código a las nuevas páginas apuntadas en el mapa (Ver sección 3.1.1). Esta acción implica que las direcciones físicas de las tareas son modificadas y remapeadas en el mapa en cada movimiento.

Para el servicio `take` el sistema observa si la celda actual tiene un cristal. De ser así, detecta si el contenedor de cristales de la tarea tiene espacio para almacenar al nuevo cristal. En caso afirmativo, guarda el cristal y retorna a la tarea el peso de este último. Si el contenedor no tiene espacio, entonces el servicio retornará cero, indicando que no se levanto ningún cristal. En el caso de que la celda no contenga ningún cristal, el servicio retornará -1.

El servicio `getId` retorna el número de tarea actual. Los identificadores de tareas son números entre 1 y 10, que indican para cada jugador el lugar que ocupa la tarea dentro de las únicas 10 que pueden ser ejecutadas en simultáneo por cada jugador.

Por último, el comportamiento de los servicios una vez ejecutados es diferente en cada caso.

- Para el servicio **move** o **take**, se debe comenzar a ejecutar la tarea **Idle** durante el resto del *quantum*. Este último comportamiento es ilustrado por la figura 5.
- Para el servicio **getId** se debe retornar inmediatamente a la tarea que lo llamo.

Cualquiera sea el caso, ninguno de los servicios debe modificar ningún registro, a excepción de los indicados anteriormente.

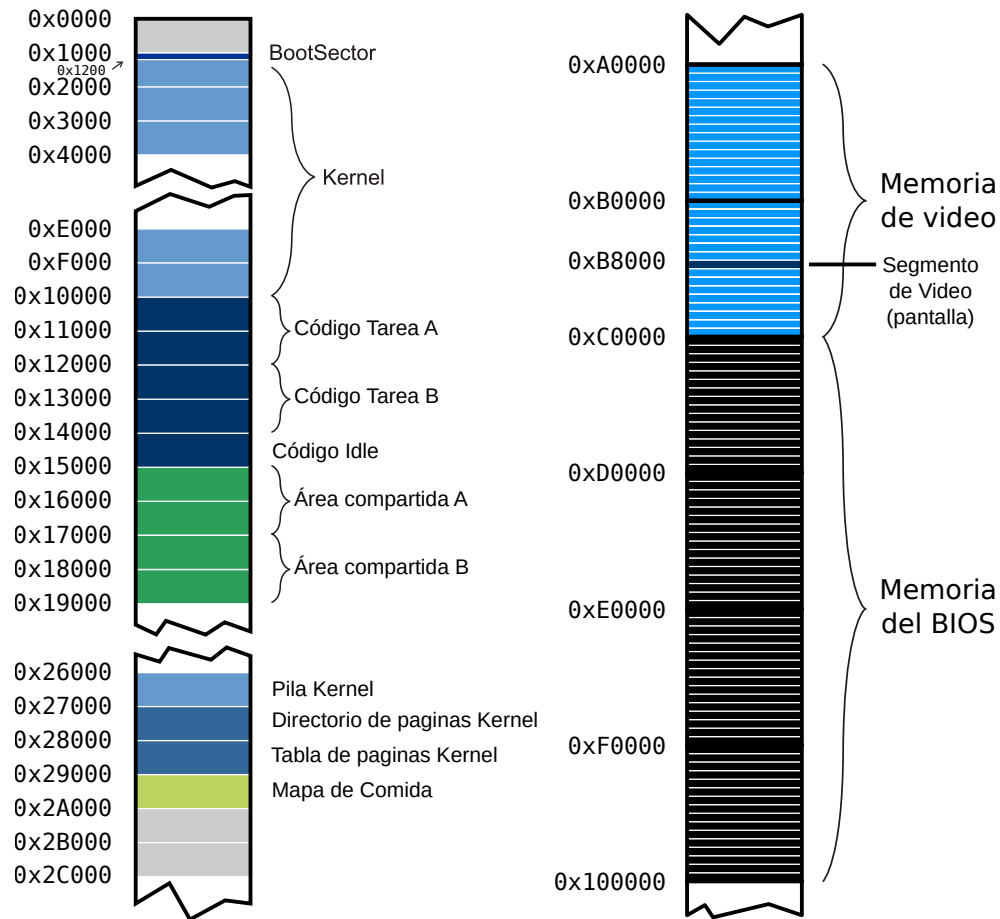


Figura 3: Mapa de la organización de la memoria física del *kernel*

3.1.1. Organización de la memoria

La memoria física estará dividida en tres áreas: *kernel*, *área libre kernel* y *mapa*. El área *kernel* corresponderá al primer MB de memoria, el *área libre kernel* a los siguientes 3 MB de memoria y el área *mapa* a los siguientes 24.375 MB.

La administración del área libre de memoria será muy básica. Se tendrá un contador de páginas a partir del cual se solicitará una nueva página. Este contador se aumentará siempre que se requiera usar una nueva página de memoria, y nunca se liberarán las páginas pedidas.

Las páginas del *área libre kernel* serán utilizadas para datos del kernel: directorios de páginas, tablas de páginas y pilas de nivel cero. Las páginas del *mapa* serán utilizadas para almacenar los códigos, datos y pila de las tareas.

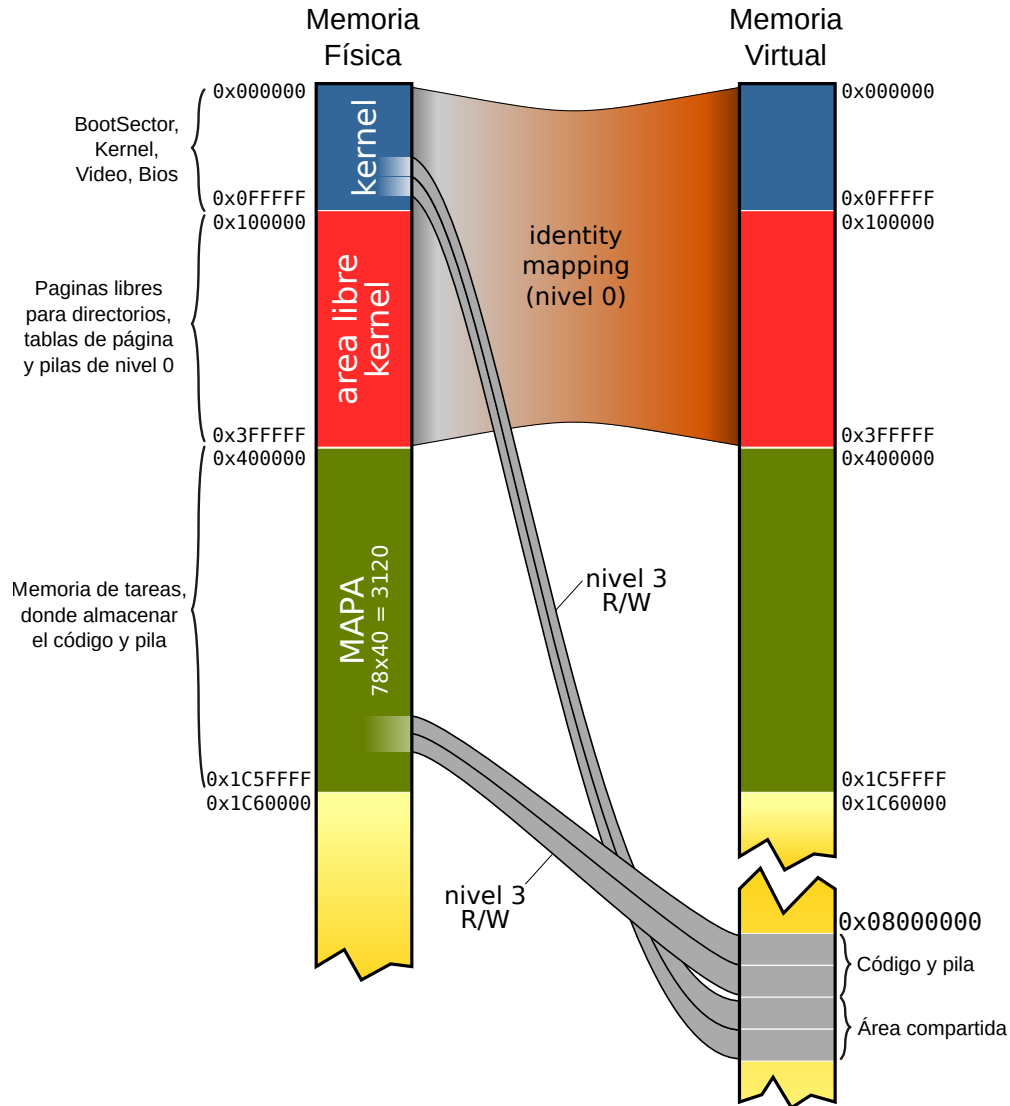


Figura 4: Mapa de memoria de la tarea

La memoria virtual de cada una de las tareas tiene mapeado el *kernel* y el *área libre kernel* con *identity mapping* en nivel 0. Sin embargo, el área *mapa* no está mapeado. Esto obliga al *kernel* a mapear el área del *mapa*, cada vez que quiera escribir en él. No obstante, el *kernel* puede escribir en cualquier posición del *área libre kernel* desde cualquier tarea sin tener que mapearla.

El código, pila y datos de las tareas estará compartido en un área de 8 KB. La copia original del mismo se encuentra almacenada en el kernel. Para construir una nueva tarea se debe realizar una copia. Las tareas además tienen mapeada un área de 8 KB compartida entre todas las tareas de un mismo jugador. La dirección física del área compartida de cada

jugador es parte del área de kernel, indicada por la figura 3. Esta área puede ser utilizada por las tareas para almacenar datos o código que pueda ser compartido entre las distintas tareas, incluso para enviar mensajes entre las instancias de las tareas que se estén ejecutando. El área compartida jamás es borrada y perdurará desde que comienza el juego. Su estado inicial no está determinado y puede no ser cero.

El código de las tareas por su parte estará mapeado en nivel 3 con permisos de lectura/escritura según indica la figura 4.

3.2. Scheduler

El sistema va a correr tareas de forma concurrente, durante un tiempo fijo denominado *quantum*. Este será determinado por un *tick* de reloj. Para lograr este comportamiento se va a contar con un *scheduler* minimal que encargará de desalojar una tarea del procesador para intercambiarla por otra en intervalos regulares de tiempo.

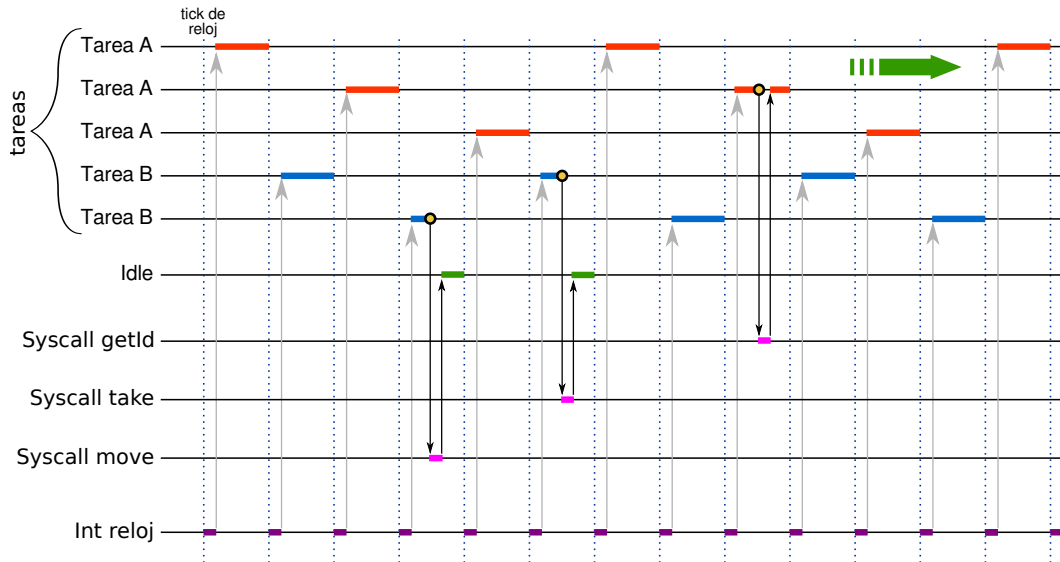


Figura 5: Ejemplo de funcionamiento del *Scheduler*

Las tareas serán ejecutadas una a una, pero separadas en dos conjuntos. En cada momento se ejecutará una tarea de uno de los jugadores y luego se ejecutará una tarea del jugador contrario. Este proceso se repetirá indefinidamente. En el caso de que no existan tareas a ejecutar, se ejecutará la tarea Idle. Esta es una tarea distinguida dentro del sistema que no realiza ninguna acción. El orden de ejecución de las tareas dentro de cada conjunto puede ser cualquiera, siempre que se asegure que todas las tareas de cada conjunto serán ejecutadas al menos una vez.

Las tareas pueden generar problemas de cualquier tipo, por esta razón se debe contar con un mecanismo que permita desalojarlas para que no puedan correr nunca más. Este mecanismo debe poder ser utilizado en cualquier contexto (durante una excepción, un acceso inválido a memoria, un error de protección general, etc.) o durante una interrupción ocasionada por haber llamado de forma incorrecta a un servicio. Cualquier acción que realice una tarea de forma incorrecta será penada con el desalojo de dicha tarea del sistema (esto en el contexto del juego equivale a que la tarea muera, liberando el lugar para una nueva tarea).

Un punto fundamental en el diseño del *scheduler* es que debe proveer una funcionalidad para intercambiar cualquier tarea por la tarea *Idle*. Este mecanismo será utilizado al momento de llamar a servicios del sistema, ya que la tarea *Idle* será la encargada de completar el *quantum* de la tarea que llamó al servicio. La tarea *Idle* se ejecutará por el resto del *quantum* de la tarea desalojada, hasta que nuevamente se realice un intercambio de tareas por la próxima que corresponda.

En la figura 5 se presenta un ejemplo con tres tareas para el jugador A y dos para el B. Se puede observar como tanto el servicio *move* como *take* son desalojados para continuar ejecutando la tarea *Idle*. El servicio *getId* en cambio, continua la ejecución de la tarea que lo llamó.

3.3. Modo debug

El sistema deberá responder a una tecla especial en el teclado, la cual activará y desactivará el modo de debugging. La tecla para tal propósito es la “y”. En este modo se deberá mostrar en pantalla la primera excepción capturada por el procesador junto con un detalle de todo el estado del procesador como muestra la figura 6. Una vez impresa en pantalla esta excepción, el juego se detendrá hasta presionar nuevamente la tecla “y” que mantendrá el modo debug pero borrará la información presentada en pantalla por la excepción. La forma de detener el juego será instantánea. Al retomar el juego se esperará hasta el próximo ciclo de reloj en el que se decidirá cuál es la próxima tarea a ser ejecutada. Se recomienda hacer una copia de la pantalla antes de mostrar el cartel con la información de la tarea.

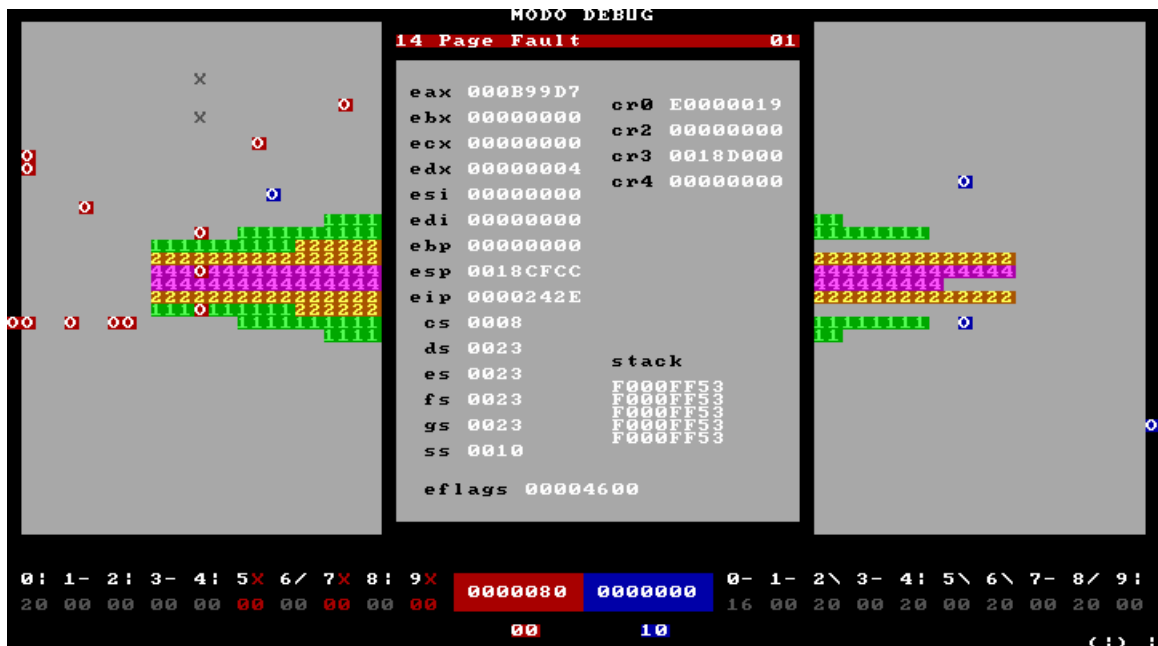


Figura 6: Pantalla de ejemplo de error

3.4. Pantalla

La pantalla presentará el tablero de 78×40 . En este mapa se indicará la posición de todas las tareas de ambos jugadores, la cantidad de puntos actuales y la cantidad de tareas restantes. Además se deberá indicar qué tareas están corriendo y cuáles lugares están libres para tareas nuevas.

La figura 7 muestra una imagen ejemplo de la pantalla indicando cuáles datos deben presentarse como mínimo. Se recomienda implementar funciones auxiliares que permitan imprimir datos en pantalla de forma cómoda. No es necesario respetar la forma exacta de presentar estos datos en pantalla. Se puede modificar la forma, no así los datos en cuestión.

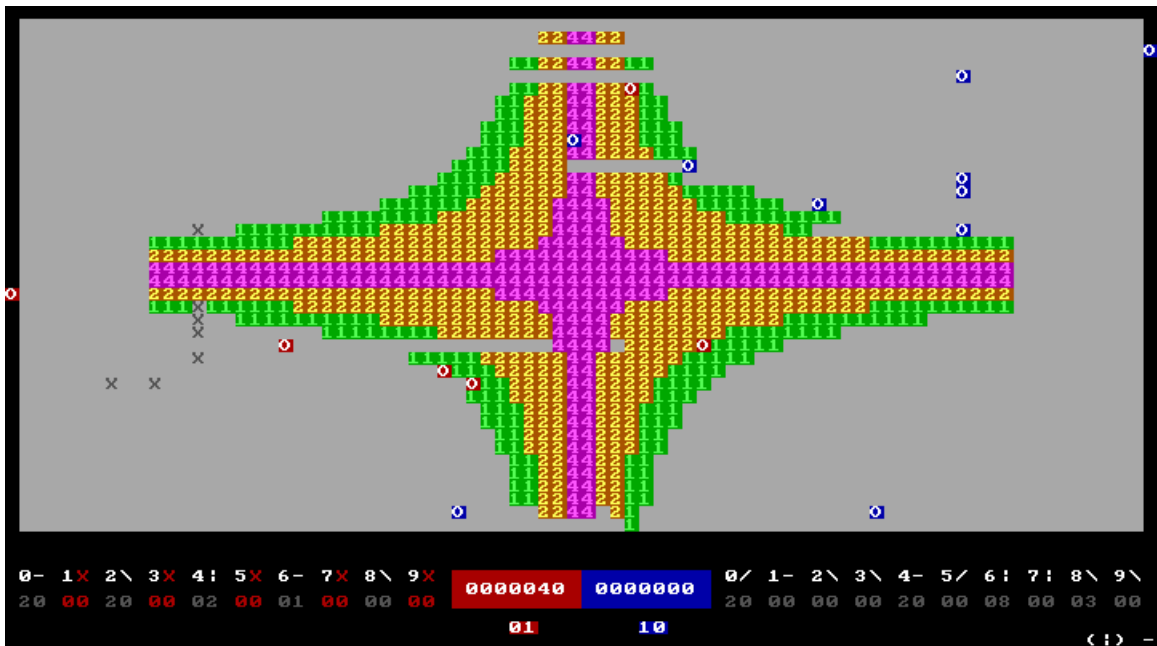


Figura 7: Pantalla de ejemplo

4. Ejercicios

4.1. Ejercicio 1

- a) Completar la Tabla de Descriptores Globales (**GDT**) con 4 segmentos, dos para código de nivel 0 y 3; y otros dos para datos de nivel 0 y 3. Estos segmentos deben direccionar los primeros 66MB de memoria. En la *gdt*, por restricción del trabajo práctico, las primeras 17 posiciones se consideran utilizadas y por ende no deben utilizarlas. El primer índice que deben usar para declarar los segmentos, es el 18 (contando desde cero).
- b) Completar el código necesario para pasar a modo protegido y setear la pila del *kernel* en la dirección `0x27000`.
- c) Declarar un segmento adicional que describa el área de la pantalla en memoria que pueda ser utilizado sólo por el *kernel*.
- d) Escribir una rutina que se encargue de limpiar la pantalla y pintar ¹ el área del tablero con algún color de fondo, junto con las barras de los jugadores según indica la sección 3.4. Para este ejercicio se debe escribir en la pantalla usando el segmento declarado en el punto anterior. Es muy importante tener en cuenta que para los próximos ejercicios se accederá a la memoria de video por medio del segmento de datos y no este último.

Nota: La **GDT** es un arreglo de `gdt_entry` declarado sólo una vez como `gdt`. El descriptor de la **GDT** en el código se llama `GDT_DESC`.

4.2. Ejercicio 2

- a) Completar las entradas necesarias en la **IDT** para asociar diferentes rutinas a todas las excepciones del procesador. Cada rutina de excepción debe indicar en pantalla qué problema se produjo e interrumpir la ejecución. Posteriormente se modificarán estas rutinas para que se continúe la ejecución, resolviendo el problema y desalojando a la tarea que lo produjo.
- b) Hacer lo necesario para que el procesador utilice la **IDT** creada anteriormente. Generar una excepción para probarla.

Nota: La **IDT** es un arreglo de `idt_entry` declarado sólo una vez como `idt`. El descriptor de la **IDT** en el código se llama `IDT_DESC`. Para inicializar la **IDT** se debe invocar la función `idt_inicializar`.

4.3. Ejercicio 3

- a) Completar las entradas necesarias en la **IDT** para asociar una rutina a la interrupción del reloj y otra a la interrupción de teclado. Además crear una entrada adicional para la interrupción de software 47.
- b) Escribir la rutina asociada a la interrupción del reloj, para que por cada *tick* llame a la función `nextClock`. La misma se encarga de mostrar cada vez que se llame, la animación de un cursor rotando en la esquina inferior derecha de la pantalla. La función `nextClock` está definida en `isr.asm`.

¹http://wiki.osdev.org/Text_UI

- c) Escribir la rutina asociada a la interrupción de teclado de forma que si se presiona cualquiera de 0 a 9, se presente la misma en la esquina superior derecha de la pantalla.
- d) Escribir la rutina asociada a la interrupción 47 para que modifique el valor de `eax` por `0x42`. Posteriormente este comportamiento va a ser modificado para atender uno de los servicios del sistema.

4.4. Ejercicio 4

- a) Escribir las rutinas encargadas de inicializar el directorio y tablas de páginas para el *kernel* (`mmu_initKernelDir`). Se debe generar un directorio de páginas que mapee, usando *identity mapping*, las direcciones `0x00000000` a `0x003FFFFFFF`, como ilustra la figura 4. Además, esta función debe inicializar el directorio de páginas en la dirección `0x27000` y las tablas de páginas según muestra la figura 3.
- b) Completar el código necesario para activar paginación.
- c) Escribir una rutina que imprima el número de libreta de todos los integrantes del grupo en la pantalla.

4.5. Ejercicio 5

- a) Escribir una rutina (`mmu_init`) que se encargue de inicializar las estructuras necesarias para administrar la memoria en el área libre de kernel y en el área libre de tareas (dos contadores de páginas libres).
- b) Escribir una rutina (`mmu_initTaskDir`) encargada de inicializar un directorio de páginas y tablas de páginas para una tarea, respetando la figura 4. La rutina debe solicitar dos páginas del área libre de tareas donde copiar el código de la tarea y mapear dicha página a partir de la dirección virtual `0x08000000` (128MB). Sugerencia: agregar a esta función todos los parámetros que considere necesarios.
- c) Escribir dos rutinas encargadas de mapear y desmapear páginas de memoria.
 - I- `mmu_mapPage(uint32_t virtual, uint32_t cr3, uint32_t phy)`
Permite mapear la página física correspondiente a `phy` en la dirección virtual `virtual` utilizando `cr3`.
 - II- `mmu_unmapPage(uint32_t virtual, uint32_t cr3)`
Borra el mapeo creado en la dirección virtual `virtual` utilizando `cr3`.
- d) A modo de prueba, construir un mapa de memoria para tareas e intercambiarlo con el del *kernel*, luego cambiar el color del fondo del primer carácter de la pantalla y volver a la normalidad. Este ítem no debe estar implementado en la solución final.

Nota: Por construcción del *kernel*, las direcciones de los mapas de memoria (`page directory` y `page table`) están mapeadas con *identity mapping*. En los ejercicios en donde se modifica el directorio o tabla de páginas, se debe que llamar a la función `tlbflush` para que se invalide la *cache* de traducción de direcciones.

4.6. Ejercicio 6

- a) Definir las entradas en la GDT que considere necesarias para ser usadas como descriptores de TSS. Mínimamente, una para ser utilizada por la *tarea inicial* y otra para la tarea *Idle*.
- b) Completar la entrada de la TSS de la tarea *Idle* con la información de la tarea *Idle*. Esta información se encuentra en el archivo `tss.c`. La tarea *Idle* se encuentra en la dirección `0x00014000`. La pila se alojará en la misma dirección que la pila del kernel y será mapeada con *identity mapping*. Esta tarea ocupa 1 página de 4KB y debe ser mapeada con *identity mapping*. Además, la misma debe compartir el mismo CR3 que el *kernel*.
- c) Construir una función que complete una TSS libre con los datos correspondientes a una tarea. El código de las tareas se encuentra a partir de la dirección `0x00010000` ocupando una página de 8kb cada una según indica la figura 3. Para la dirección de la pila se debe utilizar el mismo espacio de la tarea, la misma crecerá desde la base de la tarea. Para el mapa de memoria se debe construir uno nuevo utilizando la función `mmu_initTaskDir`. Además, tener en cuenta que cada tarea utilizará una pila distinta de nivel 0, para esto se debe pedir una nueva página del área libre de kernel a tal fin.
- d) Completar la entrada de la GDT correspondiente a la *tarea inicial*.
- e) Completar la entrada de la GDT correspondiente a la tarea *Idle*.
- f) Escribir el código necesario para ejecutar la tarea *Idle*, es decir, saltar intercambiando las TSS, entre la *tarea inicial* y la tarea *Idle*.

Nota: En `tss.c` están definidas las `tss` como estructuras TSS. Trabajar en `tss.c` y `kernel.asm`.

4.7. Ejercicio 7

- a) Construir una función para inicializar las estructuras de datos del *scheduler*.
- b) Crear la función `sched_nextTask()` que devuelve el índice en la GDT de la próxima tarea a ser ejecutada. Construir la rutina de forma devuelva una tarea de cada jugador por vez según se explica en la sección 3.2.
- c) Modificar la rutina de interrupciones `0x47` para que implemente los distintos servicios del sistema según se indica en la sección 3.1.
- d) Modificar el código necesario para que se realice el intercambio de tareas por cada ciclo de reloj. El intercambio se realizará según indique la función `sched_nextTask()`.
- e) Modificar las rutinas de excepciones del procesador para que desalojen a la tarea que estaba corriendo y ejecuten la próxima.
- f) Implementar el mecanismo de debugging explicado en la sección 3.3 que indicará en pantalla la razón del desalojo de una tarea.

4.8. Ejercicio 8 (optativo)

- a) Crear una tarea que respete las restricciones del trabajo práctico, ya que de no hacerlo no podrán ser ejecutados en el sistema implementado por la cátedra.

Debe cumplir:

- No ocupar más de 8 KB (tener en cuenta la pila)
- Tener como punto de entrada la dirección cero
- Estar compilado para correr desde la dirección 0x08000000
- Utilizar los servicios del sistema correctamente

Explicar en pocas palabras la estrategia utilizada.

- b) Si consideran que su tarea pueden hacer algo más que completar el primer ítem de este ejercicio, y se atreven a enfrentarse en batalla por juntar la máxima cantidad de cristales, entonces pueden enviar el **binario** de sus tareas a la lista de docentes indicando el nombre de la tarea.

Se realizará una competencia a fin de cuatrimestre con premios en/de chocolate para los primeros puestos.

5. Entrega

Este trabajo práctico está diseñado para ser resuelto de forma gradual. Dentro del archivo `kernel.asm` se encuentran comentarios que muestran las funcionalidades que deben implementarse para resolver cada ejercicio. También deberán completar el resto de los archivos según corresponda.

A diferencia de los trabajos prácticos anteriores, en este trabajo está permitido modificar cualquiera de los archivos proporcionados por la cátedra, o incluso tomar libertades a la hora de implementar la solución; siempre que se resuelva el ejercicio y cumpla con el enunciado. Parte de código con el que trabajan está programado en ASM y parte en C, decidir qué se utilizará para desarrollar la solución, es parte del trabajo.

Se deberá entregar un informe que describa **detalladamente** la implementación de cada uno de los fragmentos de código que fueron construidos para completar el kernel. Se espera que el informe tenga el detalle suficiente como para entender el código implementado en la solución entregada. En el caso que se requiera código adicional también debe estar descripto en el informe. Cualquier cambio en el código que proporcionamos también deberá estar documentado. Se deberán utilizar tanto pseudocódigos como esquemas gráficos, o cualquier otro recurso pertinente que permita explicar la resolución. Además se deberá entregar en soporte digital el código e informe; incluyendo todos los archivos que hagan falta para compilar y correr el trabajo en Bochs.

La fecha de entrega de este trabajo es **20/06**. Deberá ser entregado a través de un repositorio GIT almacenado en <https://git.exactas.uba.ar> respetando el protocolo enviado por mail y publicado en la página web de la materia. El informe de este trabajo debe ser incluido dentro del repositorio en formato PDF.

Ante cualquier problema con la entrega, comunicarse por mail a la lista de docentes.